Prerequisites

- NVIDIA GPU + CUDA Toolkit
- Python 3.8+

Setup

Start the virtual environment and install dependencies:

```
python -m venv venv
source venv/bin/activate

pip install -r requirements.txt
```

Usage

Web Interface

Launch the interactive dashboard to upload images, adjust preprocessing in real-time, and visualize heatmaps.

```
# Start the web server
python app.py

# Start with a specific image directory for quick selection
python app.py -i ./my_samples/
```

---

1. Oklab Color Space (Perceptual Distance)

Used in: `metrics.HumanEyeColorMetric`

The primary metric for "Human Eye" similarity uses the **Oklab** color space. Oklab is a perceptual color space designed to be simple to use and accurately predict perceived color differences (unlike sRGB or HSV). It improves upon CIELAB by correcting hue linearity issues (e.g., the "blue turns purple" problem in gradients).

Mathematical Implementation

The conversion from sRGB (input image) to Oklab involves three steps:

1. **Inverse Gamma Correction (sRGB → Linear RGB):** We assume the input C_{sRGB} is in range $[0, 1]$.

$$C_{linear} = \begin{cases} \frac{C_{sRGB}}{12.92} & C_{sRGB} \leq 0.04045 \\ \left(\frac{C_{sRGB}+0.055}{1.055}\right)^{2.4} & C_{sRGB} > 0.04045 \end{cases}$$

2. **Linear RGB → LMS (Cone Response):** Approximating the human eye's cone response using a matrix transformation M_1.

$$\begin{bmatrix} L \\ M \\ S \end{bmatrix} = \begin{bmatrix} 0.412221 & 0.536332 & 0.051445 \\ 0.211903 & 0.680699 & 0.107396 \\ 0.088302 & 0.281718 & 0.629978 \end{bmatrix} \times \begin{bmatrix} R_{lin} \\ G_{lin} \\ B_{lin} \end{bmatrix}$$

3. **Non-Linearity & LMS → Oklab:** Applying a cube root non-linearity followed by the final transformation matrix M_2.

$$\begin{bmatrix} L_{ok} \\ a_{ok} \\ b_{ok} \end{bmatrix} = \begin{bmatrix} 0.210454 & 0.793617 & -0.004072 \\ 1.977998 & -2.428592 & 0.450593 \\ 0.025904 & 0.782771 & -0.808675 \end{bmatrix} \times \begin{bmatrix} L^{1/3} \\ M^{1/3} \\ S^{1/3} \end{bmatrix}$$

Distance Calculation

To simulate human visual groupings, a **Gaussian Blur** (G_σ) is applied to the Oklab patches before comparison to suppress high-frequency noise (fabric grain). The distance between two patches P_i and P_j is calculated using a weighted Minkowski distance (p-norm):

$$D(P_i, P_j) = \left(\sum (w \cdot |P_i - P_j|)^p \right)^{1/p}$$

- L controls Lightness.
- a controls Green-Red.
- b controls Blue-Yellow.

References

- **Original Publication:** Björn Ottosson - A perceptual color space for image processing
- **Color Conversion Code:** Based on the standard Oklab implementation matrices provided in the blog above.

---

2. SSIM (Structural Similarity Index)

Used in: `metrics.SSIMMetric`

Mathematical Implementation

For two image patches x and y:

1. **Statistics:**
    - μ_x, μ_y: Mean intensity (Luminance).
    - σ_x^2, σ_y^2: Variance (Contrast).

- σ_{xy}: Covariance (Structure).

2. **Comparison Components:**

$$l(x, y) = \frac{2\mu_x\mu_y + C_1}{\mu_x^2 + \mu_y^2 + C_1}$$

$$cs(x, y) = \frac{2\sigma_{xy} + C_2}{\sigma_x^2 + \sigma_y^2 + C_2}$$

(Note: In the code, Contrast and Structure are combined into the cs term).

3. **Metric Calculation:** The code implements a customizable version allowing weights (α, β):

$$\text{SSIM}(x, y) = [l(x, y)]^\alpha \cdot [cs(x, y)]^\beta$$

4. **Distance Conversion:** Since the analyzer requires a distance matrix (where 0 is identical), the result is inverted:

$$\text{Distance} = 1 - \text{SSIM}(x, y)$$

References

- **Paper:** Wang, Z., et al. "Image quality assessment: From error visibility to structural similarity." IEEE Transactions on Image Processing (2004).
- **Explanation:** SSIM on Wikipedia

---

3. Cosine Distance

Used in: `metrics.CosineMetric`

Measures the cosine of the angle between two vectors. It determines if two image patches point in the same "direction" in high-dimensional space, regardless of their magnitude (brightness).

$$\text{Similarity} = \cos(\theta) = \frac{A \cdot B}{\|A\|\|B\|}$$

$$\text{Distance} = 1 - \cos(\theta)$$

---

4. DBSCAN Clustering

Used in: `clustering.dbscan`, `clustering.dbscan2`

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is used to identify groups of "normal" units and isolate outliers.

Algorithm Logic

1. **Core Points:** A point is a core point if it has at least `min_samples` neighbors within distance `eps`.
2. **Cluster:** Formed by connecting core points and their reachable neighbors.
3. **Noise:** Points that cannot be reached from any core point are labeled as Noise (-1). **In this application, Noise points are considered anomalies.**

Auto-Epsilon (K-Distance Graph)

To avoid manual tuning, the `find_dbscan_eps` function implements the "Knee" (or Elbow) method:

1. Calculate the distance to the k-th nearest neighbor for every point (where $k = \text{min\_samples} - 1$).
2. Sort these distances to form a curve.
3. Find the point of maximum curvature (the "knee") using the geometric distance from the line connecting the first and last points.

References

- **Paper:** Ester, M., et al. "A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise." KDD-96.
- **Knee Algorithm:** Ville Satopää, et al. "Finding a"Kneedle" in a Haystack: Detecting Knee Points in System Behavior"

---

5. Distance Transformations

Used in: `analyzer.PatchAnalyzer`

Post-processing steps applied to the distance matrix to improve clustering separation.

Sigmoid Contrast Stretch

Used to polarize the distance matrix, pushing "somewhat similar" units to 0 and "somewhat different" units to 1.

$$D_{new} = \frac{1}{1 + e^{-k(D_{old} - \mu)}}$$

- μ: The mean distance of the matrix.
- k: Steepness factor (controlled by UI).

Power Transformation

Expands the dynamic range of high distances while compressing low distances.

$$D_{new} = (D_{old})^p$$